AIDA vs LangGraph / LangChain

AIDA positioning · best-effort comparison — live source: github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-06-28 — LangGraph, LangChain, LangSmith, and Deep Agents move quickly. Re-verify against the current LangChain docs before treating capability claims here as authoritative. Sources checked: LangGraph overview, LangChain overview, and LangSmith Deployment.

TL;DR: LangGraph is a durable execution runtime for long-running, stateful agents. It is excellent at graph-shaped workflows, checkpoints, interrupts, HITL, memory, streaming, deployment, and observability when paired with LangSmith. **AIDA** is a durable coordination record for project intent: stable requirement IDs, typed relationships, code-to-spec traces, lifecycle state, leases, queue, mailbox, and MCP access across vendors. If you need to *run* a stateful agent workflow, use LangGraph. If you need agents and humans to share a long-lived record of what the project is for, which work is approved, which code implements which requirement, and how that record survives vendor/tool changes, that is AIDA's layer. They compose naturally: LangGraph runs the agents; AIDA is the substrate they coordinate around.

The comparison matters because both tools use words like **state**, **graph**, **memory**, **persistence**, and **orchestration**. Those words are true in both places, but they point at different objects. LangGraph persists **workflow execution state**. AIDA persists **program intent and coordination state**.

Where LangGraph sits in the LangChain ecosystem

LangChain's own docs draw the stack this way:

- **LangChain** is the agent framework: model/tool integrations, messages, middleware, structured output, and agent harnesses.
- **Deep Agents** is a higher-level agent harness on top of LangGraph: planning, subagents, filesystem tools, and context management.
- **LangGraph** is the low-level orchestration runtime: durable execution, streaming, human-in-the-loop, persistence, memory, and stateful workflows.
- **LangSmith** is the platform layer: tracing, evaluation, prompts, observability, deployment, and operational tooling.

So the short version is:

> LangChain builds the agent; LangGraph runs the stateful workflow; LangSmith observes and deploys it.

AIDA is not a substitute for any of those. AIDA sits one layer over: it is the project-owned record those agents read from and write back to.

What each one actually is

	LangGraph / LangChain ecosystem	AIDA
Primary job	Build and run stateful agent applications	Preserve project intent and coordination state
Core object	A graph of computation over application state	A graph of requirements, relationships, traces, and lifecycle
State lifetime	Thread/run/application lifetime, with checkpointers and stores	Project lifetime, across sessions, vendors, branches, and merges
Persistence target	Checkpointer/store such as SQLite/Postgres or LangSmith-managed infra	Git-canonical store: requirement YAML + history, with rebuildable cache
Identity	Threads, runs, assistants, messages, app-defined state keys	Stable SPEC-IDs that survive renames, merges, and vendor switches
HITL	Interrupt and inspect/modify agent state during execution	Punt/escalation/lifecycle authority recorded against a spec
Observability	LangSmith traces, evaluation, runtime metrics, deployment operations	`aida show/history/status/graph`, commits, trace comments, queue/findings/mailbox
Vendor posture	Model-agnostic framework, but one deployed application/runtime owns the workflow state	Vendor-neutral coordination record any CLI/MCP-capable agent can join
Best at	Durable execution, retry/resume, streaming, memory, production agent runtime	Stable intent, traceability, approval authority, cross-session/cross-vendor coordination

The dividing line is simple:

> LangGraph remembers where an agent workflow is. AIDA remembers what the work is for.

Where LangGraph is stronger

Use LangGraph when the problem is **agent execution**:

- You need a graph-shaped workflow mixing deterministic steps and LLM decisions.
- You need checkpoint/resume, interrupts, time travel, stores, and fault tolerance.
- You need streaming and event-level progress from a long-running agent application.
- You want human-in-the-loop edits to the execution state.
- You want production observability and evaluation through LangSmith.
- You are building an application whose core behavior is the agent workflow itself.

This is not a weak comparison for LangGraph. It is the strongest mainstream example of the orchestration layer maturing. AIDA should learn from it, and in many cases delegate execution to it rather than hand-roll equivalent runtime plumbing.

Where LangGraph does not replace AIDA

LangGraph does not, by default, answer the questions AIDA is built around:

- *Which approved requirement does this code implement?*
- *What other specs are blocked by this one?*
- *Can a fresh Codex session and a Claude session coordinate against the same project record?*
- *What changed from Draft to Done to Completed, and who had authority to move it?*
- *Which trace comments in the source still point to live requirements?*
- *If we stop using this runtime next year, do we still own the full coordination record?*

You can build some of those answers into a LangGraph application. That is the key nuance: **LangGraph is a framework powerful enough to implement pieces of AIDA's workflow.** But if the requirement graph, lifecycle semantics, traceability rules, and approval authority live only as app-specific LangGraph state, then the coordination record is still coupled to that application/runtime. AIDA's bet is that the project record should be outside any one runtime, even a good one.

How they compose

The strongest architecture is not "AIDA or LangGraph." It is:

> AIDA owns the record. LangGraph runs the workflow. LangSmith observes execution.

Concrete composition:

1. **AIDA seeds the workflow.**
   - `aida queue next` picks a spec.
   - `show_requirement` / `query_graph` provide acceptance criteria, blockers, related specs, and trace context.
   - A LangGraph workflow receives that state as input.

2. **LangGraph executes the agent process.**
   - Implementer, reviewer, and advisor nodes can be graph nodes.
   - Checkpoints allow resume after crash.
   - Interrupts implement human approval.
   - Retries, fallbacks, and circuit breakers live in the workflow runtime.
   - LangSmith traces cost, latency, state transitions, and failures.
3. **AIDA harvests durable results.**
   - Comments, findings, punts, interface changes, trace links, and status changes land back in the requirement graph.
   - Merge/PR evidence drives the lifecycle.
   - Future agents can query the result without knowing LangGraph ran the work.

The composition rule:

> Runtime facts can live in LangGraph. Project facts should land in AIDA.

Anti-patterns

Anti-pattern 1: Rebuilding LangGraph inside AIDA.
If AIDA needs checkpointed graph execution, retries, streaming, human interrupts, and runtime observability, LangGraph is exactly the kind of mature runtime to study or delegate to. AIDA's value is not a better state-machine engine.

Anti-pattern 2: Using LangGraph state as the only project memory.
If requirement identity, approval state, traceability, and cross-vendor coordination are just keys in one LangGraph app's Postgres store, the project has not escaped runtime lock-in. The execution is durable, but the coordination record is not yet a portable project asset.

Anti-pattern 3: Calling AIDA an agent framework.
AIDA is not where you should build arbitrary agent applications. It is where project-intent state, lifecycle authority, and traceability live.

Anti-pattern 4: Calling LangGraph "just orchestration."
LangGraph is a serious production runtime: persistence, HITL, memory, streaming, fault tolerance, deployment, and observability are exactly the capabilities AIDA's research docs now treat as production proof points.

When to use which

Use LangGraph / LangChain when…	Use AIDA when…
You are building an agent application	You are coordinating work on a software project
The main object is workflow state	The main object is project intent
You need checkpoint/resume and HITL inside a run	You need durable approvals, status, and traceability across runs

Use LangGraph / LangChain when…	Use AIDA when…
You need production traces and deployment	You need a program-owned record independent of the runtime
One app owns the agent loop	Many agents/vendors/humans touch the same project record
Memory is part of the app behavior	Memory is institutional context that must outlive tools

If the question is "how do I run this stateful multi-agent process reliably?", start with LangGraph. If the question is "what record do all these processes coordinate around?", start with AIDA.

Honest scope statement

LangGraph is not a superficial neighbor. It closes real gaps AIDA has identified: resumable execution, robust checkpointing, HITL interrupts, runtime memory, production traces, and deployment. If AIDA's drain becomes a richer graph execution engine, it should seriously consider LangGraph-shaped patterns rather than inventing them from scratch.

But LangGraph's strength also sharpens AIDA's positioning. As execution runtimes get better, the strategic question moves upward: **who owns the durable project record?** AIDA's answer is: the project should own it in a portable substrate. LangGraph can be the engine that runs agents against that substrate; it should not have to be the substrate itself.

See also

- agent-decision-matrix.md — build/buy/ride-native across the coordination stack.
- composition.md — recipes for composing AIDA with adjacent tools.
- ../research/2026-06-16-layered-evaluation-framework.md — L0-L5 evaluation framework.
- ../research/2026-06-28-orchestration-articles-docs-audit.md — orchestration vocabulary and production checklist applied to AIDA research.
- ../competitive-analysis/marketplace-roster.md — current landscape roster.